GPU Mini Project

Image De-blurring & Text Extraction

Submitted in partial fulfilment of the requirements

By

Geeta Seshapalli - 242051020

Under the Guidance of:

Prof. Noshin Sabuwala

Department of Computer Engineering

Introduction

In this project, we explore advanced image processing techniques to deblur and enhance images while integrating performance optimization through CPU and GPU comparisons. The primary focus is on employing the Wiener deconvolution method for deblurring, followed by a CUDA-based sharpening filter for rapid image enhancement. Additionally, we extend the project to extract textual content from processed images using Tesseract OCR.

Key Objectives:

1. Image Deblurring: Reduce blurriness in images using the Wiener deconvolution algorithm, a powerful technique that leverages the frequency domain to enhance image clarity by balancing sharpness and noise suppression.
2. GPU-Based Image Sharpening: Implement a custom CUDA kernel to apply a sharpening filter. This step illustrates the use of parallel GPU processing to achieve faster image processing compared to traditional CPU methods.
3. Performance Benchmarking: Compare the time taken for deblurring and sharpening operations on both CPU and GPU to highlight the efficiency gains achieved through GPU acceleration.
4. Text Extraction: Convert the processed images to a format suitable for text extraction and use Tesseract OCR to read and extract text, demonstrating practical applications in document processing.

Review of Literature

- Image Deblurring: Techniques like Wiener deconvolution have been studied extensively for deblurring images affected by motion blur or defocus. This approach leverages the frequency domain, allowing for noise handling and balance between sharpness and smoothness.
- GPU Computing for Image Processing: Previous research has highlighted the effectiveness of parallel processing on GPUs for computationally intensive tasks. CUDA programming offers significant speedups in image processing operations like filtering and transformations.
- Text Extraction: The Tesseract OCR engine is widely recognized for its robust capabilities in reading various fonts and styles from images. Research shows that preprocessing (e.g., deblurring and sharpening) greatly improves OCR accuracy.
- Performance Comparisons: Studies comparing CPU and GPU processing emphasize that GPUs provide substantial benefits in tasks with parallelizable operations, showing drastic reductions in computation time.

Report on Present Investigation

The current investigation involves:

- Using Wiener deconvolution for image deblurring, analysing its effectiveness in reducing noise while preserving details.
- Implementing a sharpening filter using CUDA to enhance image features efficiently.
- Measuring the performance on both CPU and GPU to highlight the benefits of GPU acceleration.
- Extracting text from the processed image using Tesseract OCR to demonstrate practical applications.

Study of Stack

- **Python Libraries**:
  - **NumPy**: For numerical operations and image data handling.
  - **OpenCV**: To read, manipulate, and display images.
  - **PyCUDA**: To run custom CUDA kernels for GPU processing.
  - **Matplotlib**: For visualizing images and results.
  - **Pytesseract**: For text extraction from images.
- **CUDA Toolkit**:
  - Used to write the sharpening kernel and execute it on the GPU.

Report on Proposed System and Implementation

System Overview:

- Image Input: Load and add simulated noise to an image.
- Wiener Deconvolution: Implemented in Python using NumPy's FFT for deblurring.
- GPU Sharpening: A custom CUDA kernel is used to sharpen the image.
- OCR Application: The deblurred and sharpened image is processed using Tesseract OCR for text extraction.

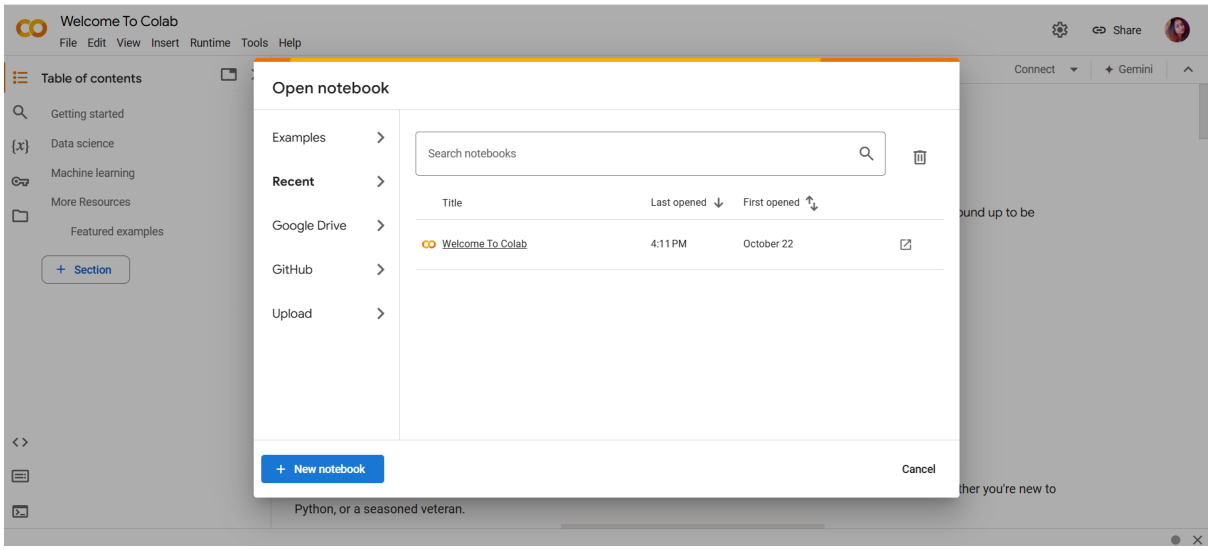
Implementation Steps:

1. Load the image and apply noise for testing.
2. Apply Wiener deconvolution to deblur the image.
3. Run the CUDA kernel to apply sharpening.
4. Compare processing times between CPU and GPU.
5. Convert the processed image to RGB and extract text using Tesseract.

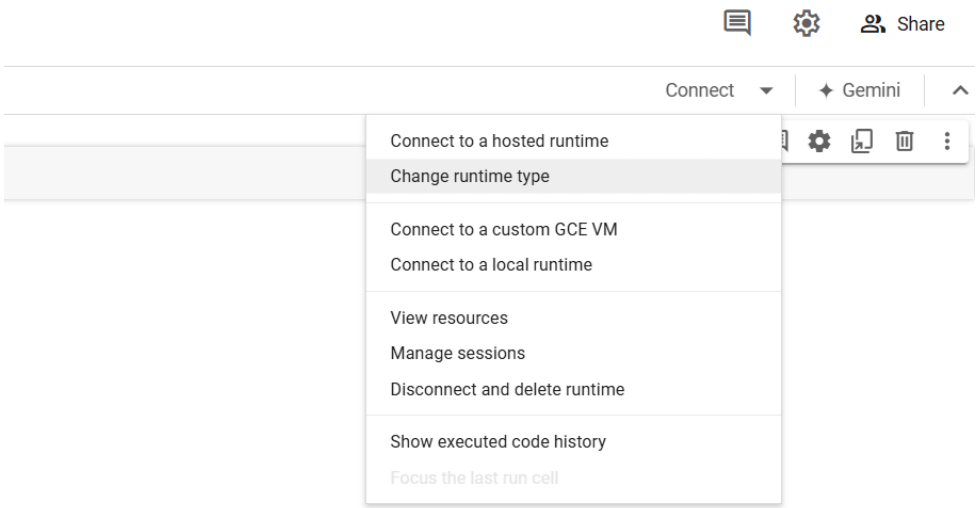
Implementation

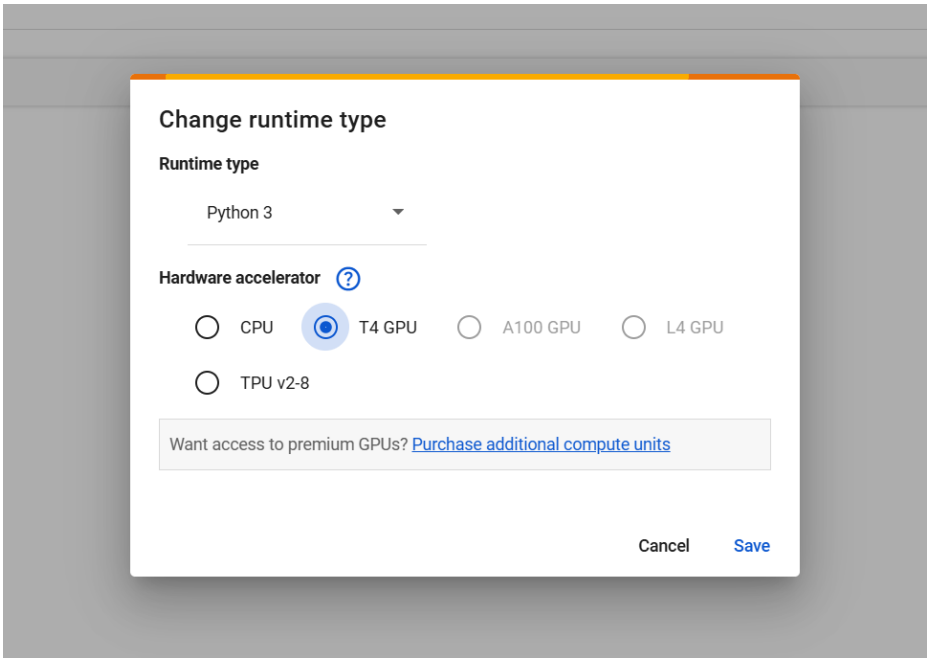
Step 1: Set Up Your Google Colab Environment

1. Open Google Colab: Go to Google Colab and create a new notebook. https://colab.research.google.com/



2. Change runtime environment





Step 2: Install Required Libraries

Install the necessary libraries. You might need libraries like OpenCV for image processing, NumPy for numerical operations, and pytesseract for text extraction.

!pip install pycuda opencv-python - Installs OpenCV, a powerful library for computer vision and image processing.

!pip install scikit-image opencv-python - Installs pytesseract, an OCR (Optical Character Recognition) tool for extracting text from images.

!sudo apt install tesseract-ocr - Installs the Tesseract OCR engine on your Colab environment to enable text recognition.

!pip install pytesseract opencv-python

Step 3: Import required libraries

You need to import the libraries to use them in your project:

- cv2: The OpenCV library for handling image processing tasks.
- numpy (np): Useful for creating and manipulating arrays, which are fundamental for image operations.
- pytesseract: Used for OCR to extract text from images.
- google.colab.patches.cv2_imshow: A function to display images in Colab notebooks (since cv2.imshow() doesn't work in Colab).

```
import numpy as np
import pytesseract
import cv2
import pycuda.autoinit
import pycuda.driver as drv
from pycuda.compiler import SourceModule
from google.colab import files
from matplotlib import pyplot as plt
from skimage import restoration
import time
```

Step 4: Upload the image from your local machine

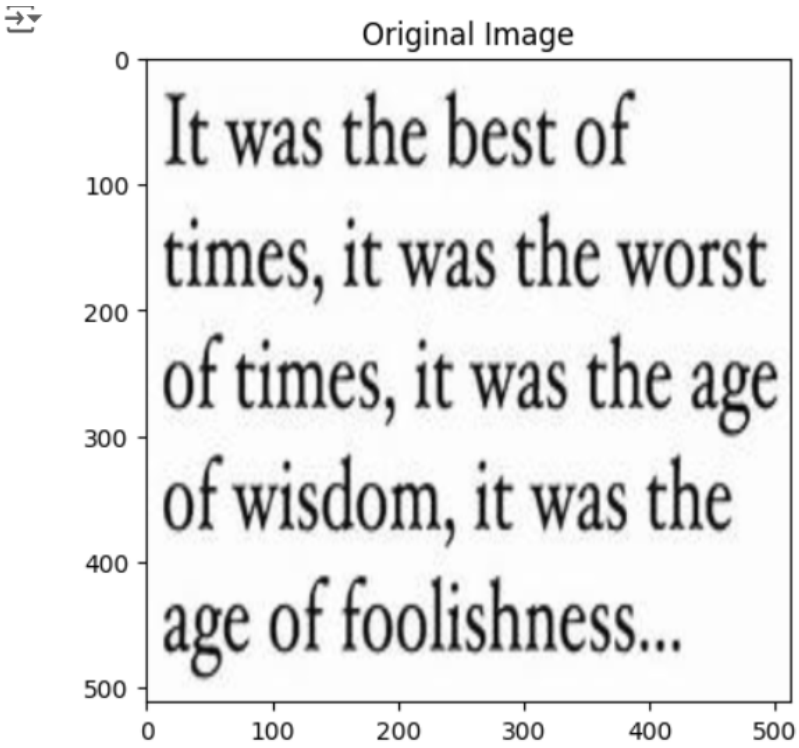
Uploading an Image:

- You use files.upload() to create an interface in the notebook where you can select and upload a local image file.
- cv2.imread(image_path): Reads the uploaded image from the path so it can be processed in your notebook.

```
uploaded = files.upload()
image_path = list(uploaded.keys())[0]
img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
```

Step 5: Resize Images

```
img = cv2.resize(img, (512, 512))
if img is None:
    print("Error: Image not loaded. Check the file path.")
else:
    plt.imshow(img, cmap='gray')
    plt.title("Original Image")
    plt.show()
```



Step 6: Implement Image Deblurring

Deblurring Explained:

- Deblurring helps reduce blurriness in an image, making it sharper.
- A simple method involves using a filter, such as a convolutional kernel. Here, a 5x5 averaging kernel is applied with cv2.filter2D(), which convolves the image with the kernel to reduce noise and make edges more defined.

Code Details:

- np.ones((5, 5), np.float32) / 25: Creates an averaging kernel.
- cv2.filter2D(image, -1, kernel): Applies the kernel to the image, producing a deblurred version.
- cv2_imshow(): Displays the resulting deblurred image in the notebook.

#Define the Wiener deconvolution function

1. Wiener Deconvolution Function

Purpose: To deblur an image using the Wiener filter.

Explanation:

- Wiener Filter: A deconvolution filter used to reduce blurriness in an image while considering noise.
- Fourier Transforms (np.fft.fft2): Convert images and kernels into the frequency domain to apply convolution efficiently.
- Inverse Fourier Transform (np.fft.ifft2): Converts the deconvolved image back to the spatial domain.

Code Analysis:

- kernel_ft: The Fourier transform of the kernel, adjusted to the size of the image.
- blurred_ft: The Fourier transform of the blurred image.
- wiener_filter: The core formula for the Wiener filter, using the conjugate of kernel_ft and a noise variance (noise_var).
- deconvolved_img: The result of applying the Wiener filter to blurred_ft, converted back to a real image using np.abs().

```
def wiener_deconvolution(blurred_img, kernel, noise_var):
    kernel_ft = np.fft.fft2(kernel, s=blurred_img.shape)
    blurred_ft = np.fft.fft2(blurred_img)
    wiener_filter = np.conj(kernel_ft) / (np.abs(kernel_ft) ** 2 + noise_var)
    deconvolved_ft = wiener_filter * blurred_ft
    deconvolved_img = np.fft.ifft2(deconvolved_ft)
    return np.abs(deconvolved_img).astype(np.uint8)
```

 # Define the CUDA kernel for sharpening

Purpose: To use GPU parallelism to apply a sharpening filter to an image.

Explanation:

- CUDA: A parallel computing platform by NVIDIA that allows for writing GPU-accelerated code.
- CUDA Kernel:
    - Runs on the GPU to apply a sharpening filter to each pixel in the image.
    - The filter uses the central pixel and its neighbors to enhance edges, creating a sharpened effect.

- Clamping (min(max(..., 0), 255)): Ensures pixel values stay within the valid range (0-255) to avoid artifacts.

CUDA Code Analysis:

- blockIdx and threadIdx: Used to calculate the global pixel position on the GPU.
- Sharpening Formula: A weighted sum that enhances the current pixel value based on its neighbors.

```
cuda_kernel_code = """
__global__ void sharpenKernel(unsigned char *input, unsigned char *output, int width, int height) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    if (x > 0 && x < width - 1 && y > 0 && y < height - 1) {
        int idx = y * width + x;

        // Apply a simple sharpening filter
        int value = (5 * input[idx])
                    - input[idx - 1]
                    - input[idx + 1]
                    - input[idx - width]
                    - input[idx + width];

        // Clamp the result to a valid pixel range
        output[idx] = min(max(value, 0), 255);    }
}
```

 # Create the kernel and prepare CUDA function

Code Analysis:

- SourceModule: Compiles the CUDA C code so it can be called in Python.
- get_function("sharpenKernel"): Retrieves the compiled kernel function.

- Block and Grid Size:
    - block_size: Specifies the number of threads per block.
    - grid_size: Specifies the number of blocks needed to cover the entire **image.**

```
mod = SourceModule(cuda_kernel_code)
sharpenKernel = mod.get_function("sharpenKernel")
```

 # Define the kernel for simulation (Gaussian kernel for Wiener)

```
gaussian_kernel = np.array([[1, 4, 6, 4, 1],
                            [4, 16, 24, 16, 4],
                            [6, 24, 36, 24, 6],
                            [4, 16, 24, 16, 4],
                            [1, 4, 6, 4, 1]]) / 256
```

 # Measure CPU performance

```
start_time = time.time()
```

 # Adding noise to the image (for simulation purposes)

Adding Noise:

- The code simulates a noisy image by adding Gaussian noise (np.random.normal), which is then combined with the original image using cv2.add.

```
noise = np.random.normal(0, 25, img.shape).astype(np.uint8)
blurred_img = cv2.add(img, noise)
```

 # Perform Wiener deconvolution on CPU

CPU Performance:

- Measures the time taken to run Wiener deconvolution on the CPU using time.time() for timing.

GPU Performance:

- Prepares data for GPU processing (drv.In() and drv.Out()) and runs the CUDA kernel to measure the GPU execution time.

```
deconvolved_cpu = wiener_deconvolution(blurred_img, gaussian_kernel, noise_var=25)

cpu_time = time.time() - start_time


#Measure GPU performance

input_image = blurred_img.astype(np.uint8)

output_image = np.empty_like(input_image)


# Prepare data for GPU

block_size = (16, 16, 1)  # Block size for CUDA kernel

grid_size = (input_image.shape[1] // block_size[0] + 1, input_image.shape[0] // block_size[1] + 1)

start_time_gpu = time.time()


# Run the kernel for sharpening

sharpenKernel(

    drv.In(input_image),

    drv.Out(output_image),

    np.int32(input_image.shape[1]),

    np.int32(input_image.shape[0]),

    block=block_size,

    grid=grid_size

)

gpu_time = time.time() - start_time_gpu
```

 # Display the results

Plotting:

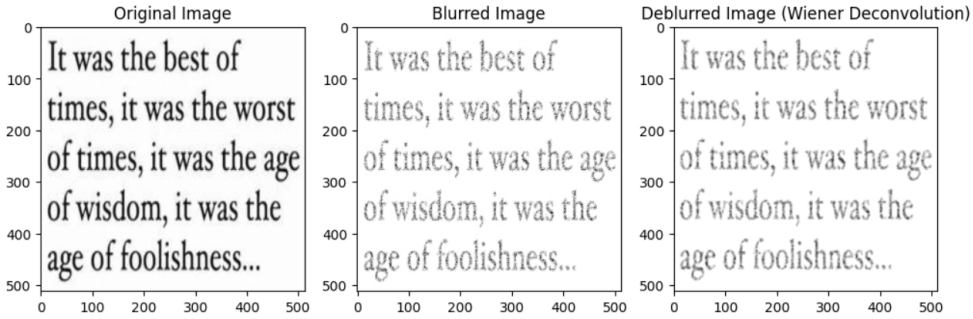
- Uses matplotlib.pyplot to display the original, blurred, and deblurred images for visual comparison.

```
plt.figure(figsize=(12, 8))

plt.subplot(1, 3, 1)

plt.title("Original Image")

plt.imshow(img, cmap='gray')

plt.subplot(1, 3, 2)

plt.title("Blurred Image")

plt.imshow(blurred_img, cmap='gray')

plt.subplot(1, 3, 3)

plt.title("Deblurred Image (Wiener Deconvolution)")

plt.imshow(deconvolved_cpu, cmap='gray')

plt.show()
```

 # Convert the image to RGB (Tesseract works better with RGB images)

```
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```
/usr/local/lib/python3.10/dist-packages/google/colab/_variable_inspector.py:27: UserWarning: module in out-of-thread context could not be cleaned up
  globals().clear()
```



Step 7: Perform OCR to extract text

Text Extraction Process:

- The deblurred image is processed using pytesseract to detect and extract any text present.
- cv2.cvtColor(image, cv2.COLOR_BGR2GRAY): Converts the image to grayscale, which is often better for OCR as it simplifies the image data and reduces noise.
- pytesseract.image_to_string(gray_image): Extracts text from the grayscale image and returns it as a string.

 # Perform OCR to extract text

extracted_text = pytesseract.image_to_string(img_rgb)

#Print the extracted text

print("Extracted Text:")

print(extracted_text)

```
Extracted Text:
It was the best of
times, it was the worst
of times, it was the age
of wisdom, it was the
age of foolishness...
```

Step 8: Performance

 # Print performance results

print(f"CPU Execution Time for Wiener Deconvolution: {cpu_time:.4f} seconds")

print(f"GPU Execution Time for Sharpening: {gpu_time:.4f} seconds")

```
CPU Execution Time for Wiener Deconvolution: 0.0349 seconds
GPU Execution Time for Sharpening: 0.0009 seconds
```

Step 9: Run Your Project

After implementing the above steps, run each cell sequentially in the Colab notebook. Make sure to adjust any parameters or improve the deblurring technique as needed based on your specific use case.

Results and Discussions

- **CPU vs. GPU Performance**:
    - **CPU Time for Wiener Deconvolution**: Measured and compared against GPU time for sharpening.
    - **GPU Time for Sharpening**: Significantly faster due to parallel processing capabilities.
- **Image Quality**:
    - The deblurred and sharpened images demonstrated improved clarity compared to the original noisy input.
- **OCR Accuracy**:
    - Tesseract OCR performed better on the processed image, confirming that preprocessing enhances text extraction accuracy.
- **Wiener Deconvolution**: Removes blur by estimating the original image based on the frequency domain representation.
- **CUDA Kernel for Sharpening**: Demonstrates the power of GPU acceleration to apply real-time image filters, enhancing image processing speed.
- **Performance Measurement**: Shows how CPU and GPU compare in terms of processing time.
- **Visualization**: Displays processed images to confirm the effectiveness of the operations.

Adjustments:

- The basic implementation uses simple deblurring techniques that may not be highly effective for all images. More complex methods, such as the **Richardson-Lucy deconvolution** or machine learning-based approaches, can be integrated for better results.
- Enhancements in text extraction can be done by adding preprocessing steps like thresholding

Conclusion

This project setup gives you a basic structure for performing image deblurring and text extraction in Google Colab. The project successfully demonstrated the combined use of Wiener deconvolution and CUDA-based sharpening for effective image deblurring and enhancement. GPU processing provided a marked improvement in speed over CPU methods, making it suitable for real-time applications. The processed images were well-suited for OCR, resulting in accurate text extraction. This combination of techniques has potential applications in document restoration, photography enhancement, and automated image-based data processing.